

David_DP&杂项

David_DP&杂项

斜率优化DP

三分

高精度

对拍

斜率优化DP

```
1  #define int long long
2  int mod=1e9+7;
3  int minProblem=1;
4  //由于dp的形式千变万化，这里仅供思路参考
5  struct pt{
6      int x,y;
7  };
8
9  //点p1,p2的斜率是否小于k
10 bool check(pt p1,pt p2,int k){
11     int c=(p2.y-p1.y);
12     int d=k*(p2.x-p1.x);
13     if(minProblem)return (p2.y-p1.y)<=k*(p2.x-p1.x);
14     else return (p2.y-p1.y)>=k*(p2.x-p1.x);
15 }
16
17 //点p1,p2的斜率是否大于p2,p3
18 bool check(pt p1,pt p2,pt p3){
19     if(minProblem)return (p2.y-p1.y)*(p3.x-p2.x)>=(p3.y-p2.y)*(p2.x-p1.x);
20     else return (p2.y-p1.y)*(p3.x-p2.x)<=(p3.y-p2.y)*(p2.x-p1.x);
21 }
22
23
24 void solve3(){
25     int n;
26     cin>>n;
27     vector<int> s(n+1);
28     vector<int> k(n+1);
29     vector<int> c(n+1);
30     vector<int> kk(n+1);
31     vector<int> ss(n+1);
32     for(int i=1;i<=n;i++){
33         cin>>s[i]>>k[i]>>c[i];
34         kk[i]=k[i]+kk[i-1];
35         ss[i]=k[i]*s[i]+ss[i-1];
36     }
37
38
39     //dp[i]=min{dp[j]+ss[j]-ss[i]+s[i]*kk[i]-s[i]*kk[j]+c[i]}
40     //dp[i]+ss[i]-s[i]*kk[i]-c[i]=dp[j]+ss[j]-s[i]*kk[j];
41     //因为s[i]已知且单调不减，使用斜率优化DP
42     //设b=y-k*x    b=dp[i]+ss[i]-s[i]*kk[i]-c[i]    y=dp[j]+ss[j]    k=s[i]
43     x=kk[j]
```

```

43
44 //如果维护max问题, 则维护上凸包, k取负数, minproblem=0
45 //如果维护min问题, 则维护下凸包, k取正数, minproblem=1
46
47 // (注意负号! 我们保证k是正数, 所以x可能为了拼凑负号变为负数)
48 //得到的新点{kk[i], dp[i]+ss[i]}插入单调队列中, 保证单调队列中两点间斜率单调递增
49 //使用一个cur维护当前符合要求的点 (要么是终点, 要么cur与cur+1两点间斜率刚好是单调队列中比k
    稍大的点)
50 //时间复杂度O(n)
51 //以下部分可以作为模板参考, 替换其中的x,y,k,b的计算方式, 只要保证k单调就可以应用
52 vector<pt> dp; //dp单调队列
53 vector<int> adp(n+1); //dp答案
54 dp.push_back({0,0});
55 int cur=0;
56 for(int i=1;i<=n;i++){
57     int k=s[i];
58     while(cur+1<dp.size()&&!check(dp[cur], dp[cur+1], k)){
59         cur++;
60     }
61
62     int x=dp[cur].x;
63     int y=dp[cur].y;
64     int b=y-x*k;
65     adp[i]=b-ss[i]+s[i]*kk[i]+c[i];
66
67     //将转移方程中x,y项中的j替换为i即可
68     int nx=kk[i];
69     int ny=adp[i]+ss[i];
70
71     while(dp.size()-2>=0&&check(dp[dp.size()-2], dp[dp.size()-1], {nx,ny})){
72         dp.pop_back();
73     }
74
75     dp.push_back({nx,ny});
76 }
77 int ans=adp[n];
78 cur=n;
79
80 //此题特判最后k为0无贡献
81 while(cur&&k[cur]==0){
82     cur--;
83     ans=min(ans, adp[cur]);
84 }
85 cout<<ans<<"\n";
86
87 }
88
89 //https://www.luogu.com.cn/problem/P6047 丝之割
90 void solve(){
91     int n,m;
92     cin>>n>>m;
93     vector<int> veca(n+1);
94     vector<int> vecb(n+1);
95     vector<int> mina(n+1, INT_MAX);
96     vector<int> minb(n+2, INT_MAX);
97     for(int i=1;i<=n;i++){
98         cin>>veca[i];
99         mina[i]=min(mina[i-1], veca[i]);
100     }
101     for(int i=1;i<=n;i++){
102         cin>>vecb[i];

```

```

103     }
104     for(int i=n;i>=0;i--){
105         minb[i]=min(minb[i+1],vecb[i]);
106     }
107     vector<pt> p(m);
108     for(int i=0;i<m;i++){
109         cin>>p[i].x>>p[i].y;
110     }
111     sort(p.begin(),p.end(),[&](const pt &a,const pt &b){
112         if(a.x==b.x){
113             return a.y>b.y;
114         }else{
115             return a.x<b.x;
116         }
117     });
118
119     vector<pt> tar;
120     for(int i=0;i<m;i++){
121         if(tar.empty()||(tar.back().y<p[i].y&&tar.back().x<p[i].x)){
122             tar.push_back(p[i]);
123         }
124     }
125     //预处理出每个点左侧最小值，右侧最小值，以及去除交叉线段
126
127     //dp[i]=dp[j]+minb(i+1)*mina(j)
128     //设b=y-k*x   b=dp[i]   y=dp[j]   k=minb(i+1)   x=-mina(j)
129     int len=tar.size();
130     vector<pt> dp;
131     int cur=0;
132     dp.push_back({-mina[tar[0].x-1],0});
133     for(int i=0;i<tar.size();i++){
134         int k=minb[tar[i].y+1];
135         while(cur+1<dp.size()&&!check(dp[cur],dp[cur+1],k)){
136             cur++;
137         }
138
139         int x=dp[cur].x;
140         int y=dp[cur].y;
141         int b=y-x*k;
142
143         //下面处理构造出的新点，主要根据x=-mina(j)和y=dp[j]，将所有的j替换为i
144         int nx=0;
145         if(i<tar.size()-1)nx=-mina[tar[i+1].x-1];    //防止末尾越界
146         int ny=b;
147
148         while(dp.size()>=2&&check(dp[dp.size()-2],dp[dp.size()-1],{nx,ny})){
149             dp.pop_back();
150         }
151         while(cur>=dp.size())cur--;
152
153         dp.push_back({nx,ny});
154     }
155     cout<<dp.back().y<<"\n";
156
157 }
158
159 //https://www.luogu.com.cn/problem/P4072 求将n个数分成m段，使得整体方差最小
160 void solve2(){
161     int n,m;
162     cin>>n>>m;
163     vector<int> vec(n+1);

```

```

164     vector<int> pre(n+1);
165     for(int i=1;i<=n;i++){
166         cin>>vec[i];
167         pre[i]=vec[i]+pre[i-1];
168     }
169     //dp[i][j]-sum(j)^2=-2sum(j)sum(k)+dp[i-1][k]+sum(k)^2
170     //设b=y-k*x    b=dp[i][j]-sum(j)^2    y=dp[i-1][k]+sum(k)^2    k=2sum(j)
x=sum(k)
171     //特别注意k*x项本身带一个负号，不要多余处理这个负号
172     //其中dp[i-1]表示上一维的答案，需要两个单调队列维护当前维的队列内容和上一维的队列内容
173     vector<vector<int>> dp(m+1,vector<int>(n+1));
174     for(int k=0;k<=n;k++){
175         dp[0][k]=0;
176     }
177     vector<pt> pdp;
178     pdp.push_back({0,0});
179     for(int i=1;i<=m;i++){
180         vector<pt> cdp;
181         int cur=0;
182         for(int j=i;j<=n;j++){
183             int k=2*pre[j];
184             while(cur+1<pdp.size()&&!check(pdp[cur],pdp[cur+1],k)){
185                 cur++;
186             }
187
188             int x=pdp[cur].x;
189             int y=pdp[cur].y;
190             int b=y-x*k;
191             dp[i][j]=b+pre[j]*pre[j];
192
193             //由于有多维，x=sum(k)和y=dp[i-1][k]+sum(k)^2无视i-1，将k替换为j
194             int nx=pre[j];
195             int ny=dp[i][j]+pre[j]*pre[j];
196
197             while(cdp.size()>=2&&check(cdp[cdp.size()-2],cdp[cdp.size()-1],
{nx,ny})){
198                 cdp.pop_back();
199             }
200
201             cdp.push_back({nx,ny});
202         }
203         swap(cdp,pdp);
204     }
205     cout<<dp[m][n]*m-pre[n]*pre[n]<<"\n";
206
207 }

```

三分

```

1  #define int long long
2  int cal(int num){return 0;}
3  void solve(){
4      int l=0,r=2e18;
5      while(r-l>2){
6          int m1=l+(r-l)/3;
7          int m2=r-(r-l)/3;
8
9          int v1=cal(m1);
10         int v2=cal(m2);
11

```

```

12         //求极大值，改为极小值修改此处符号
13         if(v1>=v2){
14             r=m2;
15         }else{
16             l=m1;
17         }
18     }
19     int ans=0;
20     for(int j=l;j<=r;j++){
21         int cur=cal(j);
22         ans=max(ans,cur);
23     }
24     cout<<ans<<"\n";
25 }

```

高精度

```

1  string removeLeadingZeros(string s){
2      int pos = s.find_first_not_of('0');
3      if(pos == string::npos) return "0";
4      return s.substr(pos);
5  }
6
7  string addBigNumbers(string num1, string num2){
8      int maxLength = max(num1.size(), num2.size());
9
10     while(num1.size() < maxLength) num1 = "0" + num1;
11     while(num2.size() < maxLength) num2 = "0" + num2;
12
13     string result = "";
14
15     int carry = 0;
16
17     for(int i = maxLength - 1; i >= 0; i--){
18         int digit1 = num1[i] - '0';
19         int digit2 = num2[i] - '0';
20
21         int sum = digit1 + digit2 + carry;
22
23         result.push_back(sum % 10 + '0');
24
25         carry = sum / 10;
26     }
27
28     if(carry){
29         result.push_back(carry + '0');
30     }
31
32     reverse(result.begin(), result.end());
33
34     return removeLeadingZeros(result);
35 }
36
37 string subtractBigNumbers(string num1, string num2){
38     bool isNegative = false;
39
40     if(num1.size() < num2.size() || (num1.size() == num2.size() && num1 < num2)){
41         swap(num1, num2);
42         isNegative = true;
43     }

```

```

44
45     int maxLength = max(num1.size(), num2.size());
46
47     while(num1.size() < maxLength) num1 = "0" + num1;
48     while(num2.size() < maxLength) num2 = "0" + num2;
49
50     string result = "";
51
52     int borrow = 0;
53
54     for(int i = maxLength - 1; i >= 0; i--){
55         int digit1 = num1[i] - '0';
56         int digit2 = num2[i] - '0';
57
58         digit1 -= borrow;
59
60         borrow = 0;
61
62         if(digit1 < digit2){
63             digit1 += 10;
64             borrow = 1;
65         }
66
67         int diff = digit1 - digit2;
68
69         result.push_back(diff + '0');
70     }
71
72     reverse(result.begin(), result.end());
73
74     result = removeLeadingZeros(result);
75
76     if(isNegative && result != "0"){
77         result = "-" + result;
78     }
79
80     return result;
81 }
82
83 string multiplyBigNumbers(string num1, string num2){
84     if(num1 == "0" || num2 == "0") return "0";
85
86     int n = num1.size();
87     int m = num2.size();
88
89     string result(n + m, '0');
90
91     for(int i = n - 1; i >= 0; i--){
92         for(int j = m - 1; j >= 0; j--){
93             int mul = (num1[i] - '0') * (num2[j] - '0');
94
95             int sum = (result[i + j + 1] - '0') + mul;
96
97             result[i + j + 1] = sum % 10 + '0';
98
99             result[i + j] += sum / 10;
100         }
101     }
102
103     return removeLeadingZeros(result);
104 }

```

```

105
106 string divideBigNumbers(string num1, int divisor){
107     string result = "";
108
109     long long cur = 0;
110
111     for(int i = 0; i < num1.size(); i++){
112         cur = cur * 10 + (num1[i] - '0');
113
114         result.push_back(cur / divisor + '0');
115
116         cur %= divisor;
117     }
118
119     return removeLeadingZeros(result);
120 }

```

对拍

```

1 // 对拍模板
2 // 文件结构: data.cpp(数据生成), solve.cpp(正解), bf.cpp(暴力), test.cpp(对拍器)
3 // 编译脚本: dpw.bat(Windows), dpL.sh(Linux)
4
5 // === data.cpp ===
6 // 数据生成器
7 mt19937_64 rd(random_device{}());
8
9 int rnd(int l, int r){
10     return l + rd() % (r - l + 1);
11 }
12
13 // 使用: cout << rnd(1, 100) << " " << rnd(1, 100);
14
15 // === solve.cpp ===
16 // 正解, 读入 n, m, 输出 n + m
17
18 // === bf.cpp ===
19 // 暴力解, 读入 n, m, 输出 ans
20
21 // === test.cpp ===
22 // 对拍器: 循环调用 data.exe, solve.exe, bf.exe, 比较输出
23 int t = 1000;
24 for(int i = 1; i <= t; i++){
25     system("data.exe > data.in");
26     system("solve.exe < data.in > solve.out");
27     system("bf.exe < data.in > bf.out");
28     if(system("fc solve.out bf.out > nul")){
29         cout << "WA";
30         break;
31     }
32     cout << i << " AC\n";
33 }
34
35 // === dpw.bat (Windows) ===
36 // g++ data.cpp -o data.exe
37 // g++ solve.cpp -o solve.exe
38 // g++ bf.cpp -o bf.exe
39 // g++ test.cpp -o test.exe
40 // test.exe
41

```

```
42 // === dpL.sh (Linux) ===  
43 // g++ data.cpp -o data  
44 // g++ solve.cpp -o solve  
45 // g++ bf.cpp -o bf  
46 // g++ test.cpp -o test  
47 // ./test
```